

CARBON EMISSION PREDICTION

PROJECT REPORT

Carbon Dioxide Emission Prediction Using Machine Learning

1. Abstract

Carbon dioxide (CO₂) emissions from vehicles are an important environmental concern because transportation contributes significantly to greenhouse gas emissions. Predicting vehicle CO₂ emissions can help understand the factors affecting emissions and support the development of more environmentally friendly vehicles.

This project presents a machine learning-based approach for predicting **CO₂ emissions in grams per kilometer (g/km)** using vehicle-related features. The dataset contains information such as vehicle manufacturer, vehicle model, vehicle class, engine-related specifications, transmission, fuel type, fuel consumption, and CO₂ emissions.

The project involves data preprocessing, categorical feature encoding, dataset splitting, machine learning model training, and model evaluation. Five regression algorithms are implemented: **Linear Regression, K-Nearest Neighbors (KNN) Regression, Decision Tree Regression, Random Forest Regression, and Support Vector Regression (SVR).**

The models are evaluated using **Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).** The models are compared to determine which algorithm provides better prediction performance for vehicle CO₂ emissions.

Keywords: CO₂ Emission, Machine Learning, Regression, Linear Regression, KNN, Decision Tree, Random Forest, SVR, Prediction.

2. Introduction

Environmental pollution and climate change are major global challenges. One of the major contributors to air pollution and greenhouse gas emissions is the transportation sector. Vehicles release carbon dioxide as a result of fuel combustion.

The amount of CO₂ released by a vehicle depends on several factors, including engine characteristics, fuel consumption, vehicle type, transmission system, and fuel type.

Machine learning provides an effective method for identifying relationships between these vehicle characteristics and CO₂ emissions. Instead of manually calculating emissions, a machine learning model can learn patterns from historical vehicle data and predict emissions for new vehicles.

In this project, different machine learning regression algorithms are used to predict the **CO₂ Emissions (g/km)** of vehicles.

3. Problem Statement

The main objective of this project is to develop a machine learning model that can predict the CO₂ emissions of vehicles based on their characteristics.

The problem can be represented as:

Input: Vehicle-related features

Output: Predicted CO₂ emissions in grams per kilometer.

The project also compares different regression algorithms to identify the model that provides the best prediction performance.

4. Objectives

The main objectives of this project are:

1. To load and understand the vehicle CO₂ emissions dataset.
 2. To perform data preprocessing.
 3. To identify and handle duplicate records.
 4. To separate input features and target variables.
 5. To convert categorical variables into numerical form using one-hot encoding.
 6. To divide the dataset into training and testing datasets.
 7. To develop multiple regression models.
 8. To predict vehicle CO₂ emissions.
 9. To evaluate model performance using MAE, MSE, and RMSE.
 10. To compare the performance of different machine learning algorithms.
 11. To identify a suitable model for CO₂ emission prediction.
-

5. Dataset Description

The project uses a vehicle CO₂ emissions dataset stored in:

```
co2.csv
```

The target variable is:

```
CO2 Emissions (g/km)
```

This represents the amount of carbon dioxide emitted by a vehicle in grams per kilometer.

Important categorical features

The code identifies the following categorical columns:

Feature	Description
Make	Vehicle manufacturer
Model	Vehicle model
Vehicle Class	Category/type of vehicle
Transmission	Type of transmission
Fuel Type	Type of fuel used

Other numerical features in the dataset are retained as numerical input variables.

6. Technologies Used

Programming Language

Python

Libraries

Library	Purpose
NumPy	Numerical calculations
Pandas	Data loading and manipulation
Matplotlib	Data visualization
Scikit-learn	Machine learning and evaluation

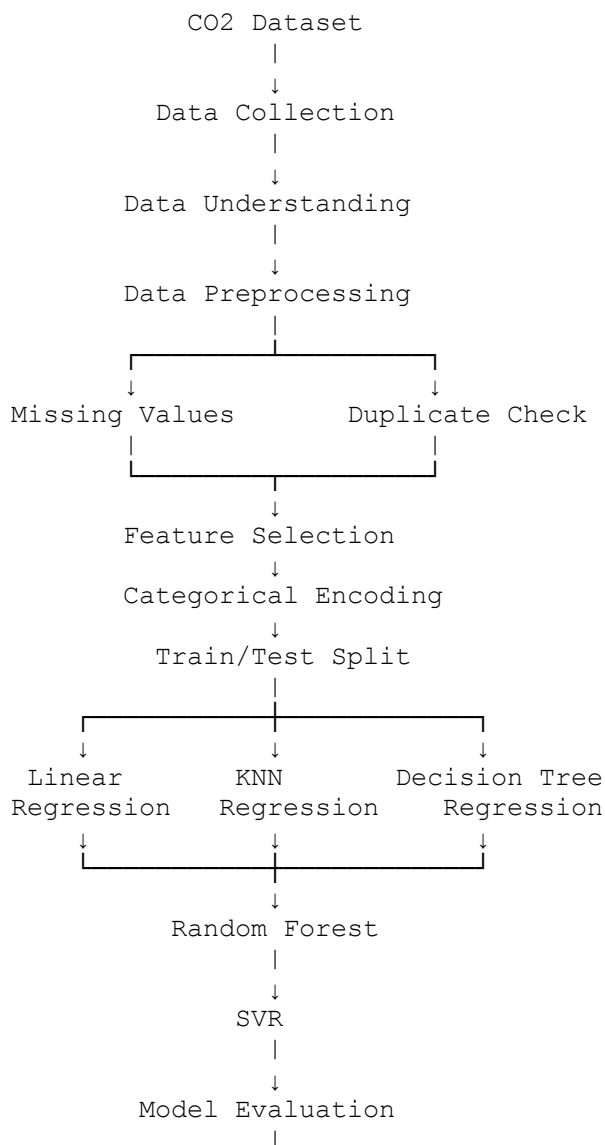
Development Environment

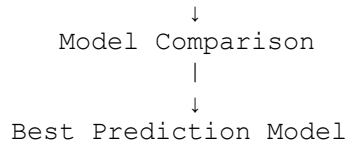
The project can be implemented using:

- Jupyter Notebook
 - Google Colab
 - VS Code
-

7. System Architecture

The overall workflow of the project is:





8. Data Preprocessing

Data preprocessing is an important step because machine learning algorithms require clean and properly formatted data.

8.1 Loading the Dataset

The dataset is loaded using Pandas:

```
df = pd.read_csv("co2.csv")
```

The first few records are displayed using:

```
print(df.head())
```

The structure and data types are examined using:

```
print(df.info())
```

8.2 Checking Missing Values

Missing values are checked using:

```
print(df.isnull().sum())
```

This helps determine whether any columns contain missing observations.

8.3 Checking Duplicate Records

Duplicate records are identified using:

```
print("Duplicates rows:", df.duplicated().sum())
```

Duplicate rows are removed using:

```
df = df.drop_duplicates()
```

This prevents repeated records from unnecessarily influencing the machine learning models.

9. Feature and Target Selection

The target variable is:

```
y = df["CO2 Emissions(g/km)"]
```

The input features are created by removing the target column:

```
x = df.drop("CO2 Emissions(g/km)", axis=1)
```

Therefore:

X → Vehicle characteristics

y → CO₂ emissions

10. Categorical Data Encoding

Machine learning algorithms generally require numerical input.

The dataset contains categorical variables such as:

```
categorical_columns = [  
    "Make",  
    "Model",  
    "Vehicle Class",  
    "Transmission",  
    "Fuel Type"  
]
```

One-hot encoding is applied:

```
x = pd.get_dummies(  
    x,  
    columns=categorical_columns,  
    drop_first=True  
)
```

This converts categorical values into numerical binary columns.

For example:

Fuel Type
Gasoline
Diesel
Electric

can be transformed into numerical indicator variables.

11. Train-Test Split

The dataset is divided into training and testing datasets:

```
x_train, x_test, y_train, y_test = train_test_split(  
    X,  
    Y,  
    test_size=0.2,  
    random_state=42  
)
```

Here:

- **80%** of the data is used for training.
- **20%** of the data is used for testing.
- `random_state=42` ensures reproducibility.

The training dataset is used to teach the model, while the testing dataset is used to evaluate how well the trained model performs on unseen data.

12. Machine Learning Models

Five regression algorithms are implemented.

12.1 Linear Regression

Linear Regression is used to model the relationship between independent variables and a continuous dependent variable.

The model is created using:

```
linear_model = LinearRegression()  
linear_model.fit(x_train, y_train)
```

Predictions are generated using:

```
y_pred_linear = linear_model.predict(x_test)
```

Linear Regression provides a simple baseline model for comparing the performance of more complex algorithms.

13. K-Nearest Neighbors Regression

KNN Regression predicts the target value based on the target values of nearby observations.

The implementation is:

```
knn_model = KNeighborsRegressor()  
  
knn_model.fit(x_train, y_train)  
  
y_pred_knn = knn_model.predict(x_test)
```

KNN can capture local patterns in the dataset. However, its performance can be affected by the scale of numerical features.

14. Decision Tree Regression

Decision Tree Regression predicts the target value by creating a tree-like structure based on feature conditions.

Implementation:

```
dt_model = DecisionTreeRegressor()  
  
dt_model.fit(x_train, y_train)  
  
y_pred_dt = dt_model.predict(x_test)
```

Decision Trees can model nonlinear relationships and are relatively easy to interpret.

15. Random Forest Regression

Random Forest Regression combines multiple decision trees to produce a more robust prediction.

Implementation:

```
rf_model = RandomForestRegressor()  
rf_model.fit(x_train, y_train)  
y_pred_rf = rf_model.predict(x_test)
```

Random Forest can capture complex nonlinear relationships and often performs well on tabular datasets.

16. Support Vector Regression

Support Vector Regression, or SVR, is an extension of Support Vector Machines for predicting continuous values.

Implementation:

```
svm_model = SVR()  
svm_model.fit(x_train, y_train)  
y_pred_svm = svm_model.predict(x_test)
```

SVR attempts to find a function that predicts the target values while maintaining an acceptable margin of error.

17. Model Evaluation

Three evaluation metrics are used in the project.

17.1 Mean Absolute Error

MAE measures the average absolute difference between actual and predicted values.

```
mae = mean_absolute_error(y_test, y_pred)
```

Lower MAE indicates better performance.

17.2 Mean Squared Error

MSE calculates the average squared difference between actual and predicted values.

```
mse = mean_squared_error(y_test, y_pred)
```

Lower MSE indicates better performance.

Because the errors are squared, large prediction errors have a greater effect on MSE.

17.3 Root Mean Squared Error

RMSE is calculated as:

```
rmse = np.sqrt(mse)
```

RMSE is useful because it expresses the prediction error in the same unit as the target variable.

Lower RMSE indicates better performance.

18. Model Comparison

Your code creates a comparison DataFrame:

```
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "KNN Regression",
        "Decision Tree Regression",
        "Random Forest Regression",
        "SVM Regression"
    ],
    "MAE": [
        mae_linear,
        mae_knn,
        mae_dt,
        mae_rf,
        mae_svm
    ],
    "MSE": [
        mse_linear,
        mse_knn,
```

```

        mse_dt,
        mse_rf,
        mse_svm
    ],

    "RMSE": [
        rmse_linear,
        rmse_knn,
        rmse_dt,
        rmse_rf,
        rmse_svm
    ]
})

print(results)

```

The resulting table allows the performance of all five models to be compared.

Results Table

Fill in the actual values produced by your program:

Model	MAE	MSE	RMSE
Linear Regression	___	___	___
KNN Regression	___	___	___
Decision Tree Regression	___	___	___
Random Forest Regression	___	___	___
SVM Regression	___	___	___

Important: Don't put invented values in your report. Use the actual output from your Jupyter Notebook/Colab.

19. Result Analysis

The models are compared using MAE, MSE, and RMSE.

The model with the **lowest MAE, MSE, and RMSE** provides lower prediction error and can generally be considered better according to those metrics.

For example, your analysis can be written as:

"Based on the experimental results, the machine learning models produced different prediction performances. Linear Regression provides a baseline prediction, while Decision Tree and Random Forest are capable of learning nonlinear relationships between vehicle characteristics

and CO₂ emissions. KNN and SVR provide alternative approaches for regression. The model with the lowest RMSE provides the most accurate predictions among the evaluated models."

After you run your code, replace this with your actual best model and values.

20. Advantages of the Proposed System

1. Automates CO₂ emission prediction.
 2. Uses multiple machine learning algorithms.
 3. Handles categorical vehicle information through encoding.
 4. Provides quantitative model evaluation.
 5. Allows comparison between different regression techniques.
 6. Can help identify patterns associated with vehicle emissions.
 7. Can be extended with optimization techniques.
-

21. Limitations

1. Prediction quality depends on the quality of the dataset.
 2. The model can perform poorly on vehicle types that are not well represented in the training data.
 3. One-hot encoding can create a large number of features when categorical columns have many unique values.
 4. KNN and SVR can be sensitive to feature scaling.
 5. The current implementation does not include hyperparameter optimization.
 6. The model predicts based on the available dataset features and does not account for every real-world factor affecting emissions.
-

22. Future Enhancement

The project can be improved in several ways.

22.1 Feature Scaling

Apply `StandardScaler` before algorithms such as KNN and SVR:

```
scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
```

```
x_test_scaled = scaler.transform(x_test)
```

22.2 Hyperparameter Optimization

Use `GridSearchCV` or `RandomizedSearchCV` to find better parameters for:

- KNN
- Decision Tree
- Random Forest
- SVR

22.3 Cross Validation

Instead of relying on one train-test split, cross-validation can provide a more reliable estimate of model performance.

22.4 Visualization

The project can include:

- Actual vs Predicted CO₂
- Model comparison chart
- Feature importance
- Error distribution
- Correlation heatmap

22.5 Model Deployment

The best model could eventually be deployed as a web application where users enter vehicle characteristics and receive a predicted CO₂ emission value.

23. Conclusion

This project demonstrates the use of machine learning for predicting vehicle CO₂ emissions. The dataset is preprocessed by checking missing values and duplicate records, removing duplicates, separating features and target variables, and converting categorical variables into numerical representations using one-hot encoding.

Five regression algorithms—**Linear Regression, KNN Regression, Decision Tree Regression, Random Forest Regression, and Support Vector Regression**—are implemented to predict CO₂ emissions.

The models are evaluated using **MAE, MSE, and RMSE**, allowing their prediction performance to be compared. The comparison helps identify the most suitable regression algorithm for the given dataset.

Overall, machine learning can provide an effective approach for estimating vehicle CO₂ emissions and can potentially support environmental analysis and the development of more efficient transportation systems.
